Bundle Update

Now that we've loaded our gems and we've made sure that we're using the right versions, things are great. For a while. But then, time passes, as it does, and new versions of gems are released. And as great as it is that Bundler is keeping our gems stabilized, we'd really like to upgrade.

The command for upgrading is `bundle update`. If it's called without any arguments, it will update all gems in the Gemfile to the most recent version, (or the most recent version allowed by the version specifier—more on that later), and will update the entire `Gemfile.lock` in response. Normally, Bundler will attempt to resolve existing version dependencies before updating but you can force it to reconsider the entire file with `bundle update --all`. You can get the same functionality by deleting the `Gemfile.lock` and re-running `bundle install`, which is a fact that you might want to keep in you back pocket for a day when Bundler's dependency management seems to be spinning its wheels.

You can update a subset of your Gemfile by listing those gems after the command, as in `bundle update rspec`. The gems you list can either be gems that are explicitly in the Gemfile itself or any dependency in the lock file. Bundler will update the listed gems to their most recent versions (again bounded by version specifiers), and will also update all dependencies of these gems. There may be some reason why you don't want to update the downstream dependencies, in which case you can attach the option `--conservative` to the end of the `bundle update` command.

Gemfile Options

The Gemfile you saw earlier in Building a Gemfile, on page 254 is a minimal file. In particular, each `gem` listing can have options that specify more information about where to get the gem in question.

Versioning

The most important information about the gem is whether to restrict its version. If you don't specify a version, Bundler will load the most recent version. However, you often don't want a complete free-for-all. If you are depending on a feature that was added to the gem along the way, you may want to specify a minimum version. If you know that a future version of the gem breaks compatibility, you'd want to add a maximum version. Or, you may just want to say "don't update this gem without me explicitly saying so."

When you add a gem to a Gemfile, you can add one or more optional version specifiers (we'll see in Writing and Packaging Your Own Code Into Gems, on page 261 that individual gems also use version specifiers to manage their dependency lists). The simplest version specifier is just an exact version number:

```
gem rspec, "3.11.0"
```

Specifying an exact version number means that Bundler won't change the version behind your back, but it also means that you lose some flexibility to respond to version dependency lists. If you want that version to be a minimum, you can use a greater than symbol.

```
gem rspec, ">= 3.11.0"
```

Similarly, a `<` or `<=` sets the version as a maximum. If you are setting a version ceiling, there's often a reason, and we recommend commenting the Gemfile to say what the reason is and